جزوه درجه یک سیستم عامل

فصل ۳ - بخش ۳: عملیات روی پردازه‌ها

به فصل سوم خوش آمدید! در بخش‌های قبلی با "پردازه" (Process) به عنوان یک مفهوم آشنا شدیم. حالا می‌خواهیم ببینیم چطور می‌توانیم این پردازه‌ها را به وجود بیاوریم (ایجاد کنیم) و چطور به کارشان پایان دهیم (خاتمه دهیم).

۳-۳-۱: ایجاد پردازه (Process Creation)

الف) مفاهیم پایه: مدل پدر-فرزندی

در سیستم عامل، پردازه‌ها مثل یک خانواده درختی هستند.

- **پردازه والد (Parent):** پردازه‌ای است که یک پردازه جدید را ایجاد می‌کند.
- **پردازه فرزند (Child):** پردازه جدیدی است که توسط والد ایجاد می‌شود.

این ساختار یک **درخت پردازه (Process Tree)** را تشکیل می‌دهد.

نکته امتحانی (PID): هر پردازه در سیستم عامل یک "کد ملی" منحصر به فرد دارد که به آن **PID (Process Identifier)** یا "شناسه پردازه" گفته می‌شود. این شناسه یک عدد صحیح است که سیستم عامل با آن پردازه‌ها را از هم تشخیص می‌دهد.

مثال در دنیای واقعی (لینوکس): در سیستم‌های لینوکس و یونیکس، "ریش سفید" یا "جد بزرگ" تمام پردازه‌ها، پردازه‌ای به نام `systemd` (یا در سیستم‌های قدیمی‌تر `init`) است. این پردازه همیشه **PID = 1** را دارد و اولین پردازه‌ای است که هنگام بالا آمدن سیستم اجرا می‌شود. بقیه پردازه‌ها (مثل دسکتاپ شما، مرورگر،...) همگی فرزندان یا نوادگان `systemd` هستند.

- **دستور `ps -el`:** لیستی از تمام پردازه‌های در حال اجرا و PID آن‌ها را به شما نشان می‌دهد.
- **دستور `pstree`:** این درخت خانوادگی پردازه‌ها را به صورت گرافیکی نمایش می‌دهد.

ب) قوانین ایجاد فرزند: چه اتفاقی می‌افتد؟

وقتی یک پردازه والد تصمیم می‌گیرد یک فرزند ایجاد کند، چند سوال مهم پیش می‌آید:

۱. مسئله منابع (پول و امکانات): فرزند جدید برای اجرا به منابع (زمان CPU، حافظه، فایل‌ها و...) نیاز دارد. این منابع را از کجا می‌آورد؟

- **گزینه ۱ (استقلال):** فرزند مستقیماً منابع مورد نیازش را از سیستم عامل (OS) می‌گیرد.
- **گزینه ۲ (وابستگی):** فرزند، منابعش را به عنوان *زیرمجموعه‌ای* از منابع والدش به ارث می‌برد (انگار والد بخشی از "سهمیه" خود را به فرزندش می‌دهد).
  - **چرا این خوب است؟** این کار یک نوع محدودیت هوشمندانه است. جلوی این را می‌گیرد که یک پردازه با ایجاد بی‌نهایت فرزند، تمام منابع سیستم را قورت بدهد و سیستم را قفل کند.

۲. مسئله اجرا (سرنوشت والد و فرزند): بعد از تولد فرزند، چه بر سر والد و فرزند می‌آید؟

- **از نظر والد:**
  - **اجرای همزمان:** والد و فرزند *همزمان* با هم به کار خود ادامه می‌دهند (مثل دو همکار).
  - **انتظار:** والد کار خود را متوقف می‌کند و *منتظر* می‌ماند تا کار فرزندش تمام شود.
- **از نظر فضای آدرس (هویت فرزند):**
  - **یک کپی برابر اصل (Clone):** فرزند یک کپی *دقیق* از برنامه و داده‌های والد است. (مثل فتوکپی)
  - **یک برنامه جدید (New Program):** فرزند یک برنامه *کاملاً جدید* را در فضای آدرس خود بارگذاری می‌کند. (انگار یک شخصیت جدید است)

این دو مورد آخر (اجرا و فضای آدرس) مهم‌ترین تفاوت بین مدل یونیکس و ویندوز هستند.

ج) پیاده‌سازی: مقایسه یونیکس (Linux) و ویندوز

این بخش **قلب** این مبحث و سوال قطعی امتحان است.

۱. مدل یونیکس/لینوکس (روش دو مرحله‌ای: "کپی کن، بعد عوض شو")

یونیکس برای ایجاد پردازه از یک جفت فراخوانی سیستمی (System Call) هوشمندانه استفاده می‌کند: `fork()` و `exec()`.

- **مرحله ۱: `fork()` (فراخوانی "فتوکپی")**
  - `fork()` یک فرزند ایجاد می‌کند که یک **کپی دقیق و کامل** از والد است. حافظه، کد، فایل‌های باز... همه چیز کپی می‌شود.
  - **نکته طلایی (راز تشخیص):** حالا ما دو پردازه یکسان داریم. چطور بفهمیم کدام والد است و کدام فرزند؟
    - `fork()` در پردازه **والد**، PID فرزند جدید (یک عدد مثبت) را برمی‌گرداند.
    - `fork()` در پردازه **فرزند**، عدد **صفر** را برمی‌گرداند.
  - به این ترتیب، پردازه‌ها می‌توانند با یک `if (pid == 0)` ساده بفهمند که "فرزند" هستند یا نه.
- **مرحله ۲: `exec()` (فراخوانی "تعویض مغز")**
  - معمولاً فرزند (که `pid == 0` را دریافت کرده) بلافاصله فراخوانی `exec()` را صدا می‌زند.
  - `exec()` فضای حافظه فعلی (که کپی والد بود) را *کامل پاک می‌کند* و یک برنامه **جدید** (مثلاً برنامه `ls` برای نمایش فایل‌ها) را در آن بارگذاری و اجرا می‌کند.
  - **مهم:** `exec` هرگز "برنمی‌گردد" (مگر اینکه خطا بدهد)، چون پردازه قبلی را کلاً نابود کرده است.
- **و والد چکار می‌کند؟ `wait()` (فراخوانی "اتاق انتظار")**
  - والدی که منتظر فرزندش بود، `wait()` را صدا می‌زند.
  - سیستم عامل والد را "می‌خواباند" تا زمانی که فرزند کارش تمام شود و `exit()` کند. بعد والد بیدار می‌شود و به کارش ادامه می‌دهد. (مثال کد شکل ۸-۳ دقیقاً همین کار را می‌کند).

۲. مدل ویندوز (روش یک مرحله‌ای: "استخدام مستقیم")

ویندوز مدل `fork/exec` را ندارد. به جای آن، یک تابع همه‌کاره به نام `CreateProcess()` دارد.

- `CreateProcess()` کار "کپی کردن" و "تعویض مغز" را *همزمان* انجام می‌دهد.
- این تابع به جای کپی کردن والد، مستقیماً یک پردازه جدید می‌سازد و از شما می‌پرسد: **"چه برنامه‌ای را می‌خواهی در این پردازه جدید اجرا کنم؟"**

- شما باید نام برنامه (مثلاً `mspaint.exe`) را به عنوان پارامتر به آن بدهید.
- `CreateProcess` بسیار پیچیده‌تر است و پارامترهای زیادی (۱۰ تا!) می‌گیرد (برخلاف `fork` که پارامتری ندارد).
- **معادل `wait` در ویندوز:** تابع `WaitForSingleObject()` است که منتظر می‌ماند تا کار پردازه فرزند تمام شود.

۳-۳-۲: ختم پردازه (Process Termination)

هر پردازه‌ای روزی به پایان می‌رسد. چطور؟

الف) روش‌های خاتمه

۱. خاتمه عادی (داوطلبانه):

- پردازه کارش تمام شده و خودش با صدا زدن فراخوانی `exit()` به سیستم عامل می‌گوید: "من تمام شدم، منابعم را پس بگیر."
- پردازه می‌تواند یک "کد وضعیت" (مثلاً یک عدد) را به والدش برگرداند تا بگوید کارش موفقیت‌آمیز بوده یا نه.

۲. خاتمه غیرعادی (اجباری):

- یک پردازه (معمولاً والد) می‌تواند پردازه دیگری (فرزندش) را به زور خاتمه دهد (مثلاً با `TerminateProcess` در ویندوز).
- **دلایل خاتمه اجباری توسط والد:**
  1. **استفاده بیش از حد منابع:** فرزند وحشی شده و دارد تمام حافظه را مصرف می‌کند.
  2. **کار دیگر لازم نیست:** وظیفه‌ای که به فرزند سپرده شده بود، دیگر مورد نیاز نیست.
  3. **والد در حال خروج است:** والد خودش دارد `exit` می‌کند. در برخی سیستم‌ها، اگر والد بمیرد، فرزندانش هم باید بمیرند. به این پدیده **ختم آبشاری (Cascading Termination)** می‌گویند.

ب) مفاهیم بعد از مرگ (سوال ۱۰۰٪ امتحانی: زامبی و یتیم)

این دو مفهوم بسیار مهم هستند و همیشه با هم مقایسه می‌شوند.

۱. پردازه زامبی (Zombie Process) (پردازه نیمه‌جان)

- **تعریف:** پردازه‌ای که کارش تمام شده و `exit()` را صدا زده، **اما** والدش هنوز `wait()` را برای دریافت "کد وضعیت" (نتیجه کار) فرزند صدا نزده است.
- **وضعیت:** پردازه مُرده است، تمام منابعش (حافظه، CPU و...) آزاد شده‌اند، **به جز** یک ورودی کوچک در "جدول پردازه‌های سیستم عامل".
- **چرا این ورودی باقی می‌ماند؟** تا "کد وضعیت" خروج را برای والد نگه دارد.
- **تشبیه:** مثل "روح سرگردان" است که منتظر است پیام آخرش را به والدینش برساند و بعد محو شود.
- **چطور پاک می‌شود؟** به محض اینکه والد `wait()` را صدا بزند، سیستم عامل آن کد وضعیت را به والد می‌دهد و آن ورودی کوچک (زامبی) را از جدول پاک می‌کند.

۲. پردازه یتیم (Orphan Process)

- **تعریف:** پردازه‌ای که در حال اجرا است (زنده است)، **اما** والدش ***قبل‌از*** صدا زدن `wait()` خودش خاتمه یافته (مرده است).
- **وضعیت:** پردازه زنده و فعال است، اما "بی‌سرپرست" شده.
- **مشکل کجاست؟** اگر این پردازه یتیم روزی `exit` کند، دیگر والدی وجود ندارد که `wait()` را برایش صدا بزند و آن را "جمع کند". در نتیجه به یک زامبی ابدی تبدیل می‌شود که جدول پردازه‌ها را پر می‌کند.
- **راه‌حل سیستم عامل (فرزندخواندگی):**
  - سیستم عامل (یونیکس/لینوکس) بسیار هوشمند است. به محض اینکه پردازه‌ای یتیم می‌شود، **پردازه شماره ۱ (`systemd`)** آن را به "فرزندخواندگی" قبول می‌کند.
  - `systemd` مثل یک پدرخوانده مهربان، به صورت متناوب `wait()` را صدا می‌زند و تمام فرزندخوانده‌هایش را که مرده‌اند (زامبی شده‌اند) جمع‌آوری می‌کند. این کار سیستم را تمیز نگه می‌دارد.

۳-۳-۳: مدیریت پردازه در اندروید (یک مثال خاص)

سیستم عامل‌های موبایل (مثل اندروید) با محدودیت شدید منابع (خصوصاً حافظه) روبرو هستند. به همین دلیل، اندروید مجبور است گاهی اوقات پردازه‌ها را برای آزاد کردن حافظه "بکُشد".

اما کدام‌ها را؟ اندروید یک **سلسله مراتب اهمیت** دارد و پردازه‌ها را از "کم‌اهمیت‌ترین" شروع به کشتن می‌کند:

1. **پردازه پوچ (Empty):** هیچ کاری نمی‌کند. (اولین کاندیدای مرگ)
2. **پردازه پس‌زمینه (Background):** در پشت صحنه کاری می‌کند که کاربر نمی‌بیند.
3. **پردازه سرویس (Service):** کاری در پس‌زمینه می‌کند که برای کاربر ***ملموس*** است (مثل پخش موسیقی).
4. **پردازه قابل رؤیت (Visible):** در صفحه دیده می‌شود، اما کاربر مستقیماً با آن کار نمی‌کند (مثلاً یک ویدئوی کوچک در کنار صفحه).
5. **پردازه پیش‌زمینه (Foreground):** پردازه‌ای که کاربر ***دقیقاً*** در حال کار با آن است. (اندروید به ندرت و فقط در شرایط بحرانی این را می‌کُشد).

اندروید با این روش، تجربه کاربری را روان نگه می‌دارد و اطمینان حاصل می‌کند که پردازه‌های مهم قربانی کمبود حافظه نمی‌شوند.